

Assignment: Systematischer Modellvergleich

mit klassischen Modellen und neuronalen Netzen – Mehrklassen-Klassifikation

Kurs	Maschinelles Lernen mit Python, scikit-learn und Keras
Hochschule	DHBW Stuttgart
Semester	Sommersemester 2026
Gruppengröße	5 Gruppen mit jeweils 4 Personen pro Gruppe
Abgabe	Spätestens bis 10. August 2026

1. Aufgabenüberblick

Jede Gruppe erhält einen realen Datensatz aus der Praxis. Ihre Aufgabe ist es, eine vollständige Machine-Learning-Pipeline durchzuführen und dabei fünf Modelle systematisch zu vergleichen: drei klassische Modelle aus dem Kurs sowie zwei Varianten eines neuronalen Netzes (MLP).

Es handelt sich um eine Mehrklassen-Klassifikation: Die Zielvariable hat mehr als zwei Klassen. Achten Sie deshalb durchgängig auf die korrekte Behandlung mehrerer Klassen – bei der Kodierung der Zielvariable, bei der Wahl der Verlustfunktion, bei den Metriken und bei der Interpretation der Konfusionsmatrix.

Ziel ist nicht das bestmögliche Ergebnis, sondern ein methodisch sauberes, nachvollziehbares und kritisch reflektiertes Vorgehen. Alle Entscheidungen müssen begründet werden.

2. Zu vergleichende Modelle

Jede Gruppe vergleicht exakt die folgenden fünf Modelle in der angegebenen Reihenfolge:

#	Modell	Typ	Bezug zum Kurs
1	Logistic Regression (multinomial)	Klassisch	Kapitel 4
2	Decision Tree, Random Forest oder XGBClassifier	Klassisch	Kapitel 5 – wähle ein geeignetes Modell
3	k-Nächster-Nachbar oder Support Vector Machine	Klassisch	Kapitel 7, 8 – wähle ein geeignetes Modell
4	Neuronales Netz Variante A	Neuronal	Kapitel 9
5	Neuronales Netz Variante B	Neuronal	Kapitel 9

Architekturen der neuronalen Netze – Startkonfiguration (`n_classes` = Anzahl der Klassen)

Variante A:

Input → Dense(64, relu) → Dense(`n_classes`, softmax)

Variante B:

Input → Dense(64, relu) → Dropout(0.3) → Dense(32, relu) → Dense(`n_classes`, softmax)

Die Ausgabeschicht hat so viele Neuronen wie Klassen und nutzt softmax. Entsprechend ist die Verlustfunktion `sparse_categorical_crossentropy` (Labels als Integer-Codes) bzw. `categorical_crossentropy` (Labels als One-Hot).

3. Teilaufgaben

Bearbeiten Sie alle sieben Schritte vollständig und in der angegebenen Reihenfolge. Dokumentieren Sie jede Entscheidung direkt im Notebook mit Markdown-Zellen.

Schritt 1 – Datenanalyse

- Datensatz laden, Form, Datentypen und fehlende Werte überblicken
- Verteilung der Zielvariable über alle Klassen dokumentieren (absolut und prozentual) – wie ausgewogen sind die Klassen?
- Mindestens 3 aussagekräftige Visualisierungen erstellen:
 - Verteilung numerischer Features (Histogramme)
 - Kategorische Features (Balkendiagramme), inkl. Klassenverteilung der Zielvariable
 - Korrelationsmatrix der numerischen Features
 - Zusammenhang zwischen Features und Zielklassen
- Welche Features erscheinen besonders relevant? Begründen Sie Ihre Einschätzung.

Schritt 2 – Preprocessing und Datenvorbereitung

- Fehlende Werte identifizieren, behandeln und Entscheidung begründen
- Kategorische Features kodieren (One-Hot oder Label Encoding – mit Begründung)
- Zielvariable kodieren: Klassentexte mit LabelEncoder in Integer-Codes umwandeln und das Mapping (Klasse → Code) dokumentieren
- Stratifizierten Train / Validation / Test-Split durchführen (70 / 15 / 15) – bei mehreren, teils kleinen Klassen ist die Stratifizierung besonders wichtig
- Features skalieren – welche Features und warum?
- Darf der Scaler auf dem gesamten Datensatz gefittet werden? Begründen Sie Ihre Antwort.

Schritt 3 – Klassische Modelle

- Alle drei Modelle mit Default-Parametern auf den Trainingsdaten trainieren
- Evaluation mit 5-facher stratifizierter Kreuzvalidierung
- Für jedes Modell: Accuracy, F1-Score (weighted) und falls sinnvoll weitere Metriken berechnen.
- Ergebnisse in die Vergleichstabelle (Schritt 6) eintragen
- Kurzen Kommentar zu jedem Modell im Notebook verfassen

Freiheitsgrade bei der Modellbildung (gilt für Variante A und B)

Die vorgegebenen Architekturen und Optimizer sind Ihre Startkonfiguration, nicht das Endziel. Trainieren Sie zunächst exakt die vorgegebene Variante.

Lesen Sie dann die Lernkurven: Zeigt sich das Modell als ungeeignet (z. B. vorzeitiger Abbruch durch Early Stopping, kein Lernfortschritt, Konvergenz auf Zufallsniveau), dürfen und sollen Sie eingreifen – über Hyperparameter wie Lernrate, patience oder Regularisierung oder die Architektur.

Jede Änderung ist in einer Markdown-Zelle zu dokumentieren und zu begründen. Bewertet wird nicht, dass die Startvariante perfekt läuft, sondern dass Sie ihre Schwächen erkennen und begründet darauf reagieren.

Schritt 4 – Variante A: Einfaches neuronales Netz

- Architektur (Startkonfiguration): Input → Dense(64, relu) → Dense(n_classes, softmax)
- Optimizer: Adam | Loss: sparse_categorical_crossentropy | max. 100 Epochen | batch_size: 32
- Ausgabeschicht: softmax mit so vielen Neuronen wie es Klassen gibt
- Early Stopping: monitor='val_loss', patience=5, restore_best_weights=True
- Überprüfe die Modellarchitektur auf Eignung: Falls ungeeignet, wähle eine optimierte Variante.
- Lernkurven (Loss und Accuracy) plotten und kommentieren:
 - Gibt es Hinweise auf Overfitting oder Underfitting?
 - In welcher Epoche hat Early Stopping abgebrochen?
- Accuracy und F1 (weighted) auf dem Validierungsset dokumentieren

Schritt 5 – Variante B: Tieferes Netz mit Regularisierung

- Architektur (Startkonfiguration): Input → Dense(64, relu) → Dropout(0.3) → Dense(32, relu) → Dense(n_classes, softmax)
- Optimizer: SGD (statt Adam) | Loss: sparse_categorical_crossentropy | max. 100 Epochen | batch_size: 32
- Early Stopping identisch zu Variante A
- Überprüfe die Modellarchitektur auf Eignung: Falls ungeeignet, wähle eine optimierte Variante.
- Lernkurven plotten und mit Variante A vergleichen:
 - Was verändert der Dropout in den Lernkurven?
 - Warum konvergiert SGD anders als Adam?
- **Zentrale Frage:** Warum ist Variante B nicht automatisch besser als Variante A?

Schritt 6 – Modellvergleich, Optimierung und finale Evaluation

- Vergleichstabelle aller Modelle auf dem Validierungsset ausfüllen:

Modell	Typ	Accuracy	F1 (weighted)	Notizen
Logistic Regression	Klassisch	___	___	
Decision Tree / RF	Klassisch	___	___	
k-NN / SVM	Klassisch	___	___	
MLP Variante A	Neural Net	___	___	
MLP Variante B	Neural Net	___	___	

- Hyperparameter-Optimierung für das beste Modell:
 - Klassische Modelle: GridSearchCV (mind. 2 Parameter)
 - Neuronale Netze: KerasTuner (mind. 2 Parameter)
- Finale Evaluation des optimierten Modells exklusiv auf dem Testset – nur einmal!
- Konfusionsmatrix (n×n) des besten Modells ausgeben und interpretieren:
 - Welche Klassen werden am häufigsten miteinander verwechselt?
 - Welche Klasse wird am schlechtesten erkannt (niedrigster Recall)?

- Was bedeutet das im Praxiskontext Ihres Datensatzes?
- Ergänzend den `classification_report` ausgeben (Precision, Recall, F1 je Klasse) – er zeigt Schwachstellen, die die Accuracy allein verbirgt.

Schritt 7 – Reflexion und Fazit

- Welches Modell empfehlen Sie für den Produktiveinsatz – und warum?
- Was hat Sie im Verlauf der Arbeit überrascht?
- Wo sehen Sie Grenzen des besten Modells im Praxiskontext?
- Was würden Sie mit mehr Zeit, mehr Daten oder mehr Ressourcen anders machen?
- Bonusfrage (optional): Hätte ein CNN oder Transfer Learning hier geholfen – warum (nicht)?

4. Abgabe und Deliverables

Bitte reichen Sie folgende Dokumente bis zum 10. August 2026 ein:

1. Google Colab Notebook (.ipynb) – vollständig ausgeführt, alle Zellen mit Markdown-Kommentaren versehen
2. PDF-Export des Notebooks – direkt aus Colab: Datei → Drucken → Als PDF speichern
3. Management Summary – Zusammenfassung der wesentlichen Ergebnisse

5. Hinweise zur KI-Nutzung

KI-Tools sind ausdrücklich erlaubt. ChatGPT, Claude, GitHub Copilot und ähnliche Werkzeuge dürfen für Code, Debugging und Erklärungen eingesetzt werden. Es gelten folgende Regeln:

- Jeden KI-generierten Codeblock im Notebook kennzeichnen: # Quelle: [Tool-Name], z. B. # Quelle: ChatGPT
- KI-generierter Code muss vollständig verstanden werden. Bei Rückfragen muss jeder Code-Abschnitt erklärt werden können.
- Interpretationen, Entscheidungsbegründungen und Reflexionen müssen eigenständig verfasst werden. KI darf als Diskussionspartner dienen, nicht als Autor.
- Die finale Empfehlung im letzten Schritt trifft die Gruppe – nicht die KI.
- Wer nicht erklären kann, was sein Code tut, hat ihn nicht verstanden. Das ist kein KI-Problem, sondern ein Lernproblem.

6. Praktische Tipps

- Beginnen Sie mit der Datenanalyse – sie gibt Ihnen das nötige Verständnis für alle folgenden Entscheidungen.
- Kommentieren Sie jede Designentscheidung direkt im Notebook mit einer Markdown-Zelle. Später erinnern Sie sich nicht mehr, warum Sie etwas getan haben.
- Nutzen Sie Google Colab mit GPU-Laufzeit (Laufzeit → Laufzeittyp ändern → T4 GPU) für die Keras-Modelle.
- Bei mehreren, teils kleinen Klassen: immer mit stratify=y splitten, damit jede Klasse in allen Teilmengen vertreten ist.
- Schauen Sie sich die Konfusionsmatrix genau an – sie zeigt, welche Klassen verwechselt werden, und sagt oft mehr als die Accuracy allein.
- Faustregel: Das Testset wird exakt einmal angefasst – nach allen Entscheidungen.

Didaktischer Hinweis

Achten Sie bei der Bewertung des „besten“ Modells nicht nur auf die Accuracy. Bei imbalancierten Mehrklassen-Problemen kann ein Modell eine hohe Gesamtgenauigkeit erreichen und trotzdem einzelne (kleine) Klassen kaum erkennen. Der F1-Score (weighted) und der classification_report machen das sichtbar.

Gruppe A: Datensatz „Wine Quality“

Datensatz	Wine Quality – Rotwein (Qualitäts-Einstufung anhand physikalisch-chemischer Messwerte)
Quelle	kagglehub: uciml/red-wine-quality-cortez-et-al-2009 (Kaggle: kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009)
Umfang	1.599 Samples 11 Features (alle numerisch)
Aufgabe	Mehrklassen-Klassifikation der Qualität (Zielvariable quality, Scores 3 bis 8 – 6 Klassen)
Besonderheit	Stark imbalanciert; die Randklassen 3 und 8 sind extrem selten (unter 20 Fälle) und kaum verlässlich erkennbar. Benachbarte Scores überlappen stark. Eine Vergrößerung zu niedrig/mittel/hoch ist als begründeter Freiheitsgrad denkbar.

Gruppe B: Datensatz „Mobile Price Range“

Datensatz	Mobile Price Range (Einstufung der Preisklasse anhand technischer Spezifikationen)
Quelle	kagglehub: iabhishekoofficial/mobile-price-classification (Datei train.csv – test.csv enthält keine Labels)
Umfang	2.000 Samples 20 Features (alle numerisch, mehrere binär)
Aufgabe	Mehrklassen-Klassifikation in 4 Preisklassen (Zielvariable price_range): 0 = niedrig, 1 = mittel, 2 = hoch, 3 = sehr hoch
Besonderheit	Exakt ausgewogen (je 500 pro Klasse). Die Klassen sind geordnet (ordinal). Keine kategorischen Text-Features; die Zielvariable liegt bereits als Integer 0–3 vor. Wichtig: Nur train.csv enthält Labels.

Gruppe C: Datensatz „Fetal Health Classification“

Datensatz	Fetal Health Classification (CTG-basierte Einstufung des fetalen Zustands)
Quelle	kagglehub: andrewmvd/fetal-health-classification (Kaggle: kaggle.com/datasets/andrewmvd/fetal-health-classification)
Umfang	ca. 2.126 Samples 21 Features (alle numerisch)
Aufgabe	Mehrklassen-Klassifikation in 3 Klassen (Zielvariable fetal_health): 1 = Normal, 2 = Suspect, 3 = Pathological
Besonderheit	Stark imbalanciert (ca. 78 / 14 / 8 %): Die Accuracy allein ist trügerisch, entscheidend ist der Recall der kleinen Klasse Pathological. Keine kategorischen Features; die Zielvariable liegt bereits numerisch vor.

Gruppe D: Datensatz „Obesity Levels“

Datensatz	Obesity Levels (Schätzung des Adipositas-Grades nach Ernährung und körperlicher Verfassung)
Quelle	kagglehub: ankurbajaj9/obesity-levels (Kaggle: kaggle.com/datasets/ankurbajaj9/obesity-levels)
Umfang	ca. 2.111 Samples 16 Features (numerisch und kategorisch)
Aufgabe	Mehrklassen-Klassifikation in 7 Gewichtskategorien (Zielvariable NObeyesdad): Insufficient_Weight, Normal_Weight, Overweight_Level_I, Overweight_Level_II, Obesity_Type_I, Obesity_Type_II, Obesity_Type_III
Besonderheit	Sowohl Features als auch die Zielvariable müssen kodiert werden. 7 Klassen mit leichter Imbalance. Diese Besonderheiten sind Teil der Aufgabe.

Gruppe E: Datensatz „Body Performance“

Datensatz	Body Performance (Einstufung der körperlichen Leistungsfähigkeit anhand von Fitness-Messwerten)
Quelle	kagglehub: kukuroo3/body-performance-data (Kaggle: kaggle.com/datasets/kukuroo3/body-performance-data)
Umfang	ca. 13.400 Samples 11 Features (überwiegend numerisch, gender kategorisch)
Aufgabe	Mehrklassen-Klassifikation in 4 Leistungsklassen (Zielvariable class): A, B, C, D
Besonderheit	Großer, ausgewogener Datensatz – die Modellwahl in Schritt 3 sollte die Rechenzeit mitberücksichtigen. Nur ein kategorisches Feature (gender); die vier Leistungsklassen sind annähernd gleich besetzt.